HDOC - 66th day

MATRIX MULTIPLICATION USING RECURSION

Name: Piyush Negi **SAP ID:** 590042185

Problem Statement

Write a recursive C program to multiply two square matrices. Do not use iterative loops for the multiplication logic.

The program must read the size and elements of two square matrices, multiply them recursively, and display the resultant matrix.

Algorithm

1. Start.

2. Read the order n of the two square matrices.

3. Read all elements of the first matrix A recursively.

4. Read all elements of the second matrix B recursively.

5. Initialize every element of result matrix C to 0.

6. Call multiply(A, B, C, n, 0, 0, 0).

7. For each C[i][j], recursively calculate the sum of A[i][k] * B[k][j] for k = 0 to n - 1.

8. After one element is completed, recursively move to the next column; after one row is completed, move to the next row.

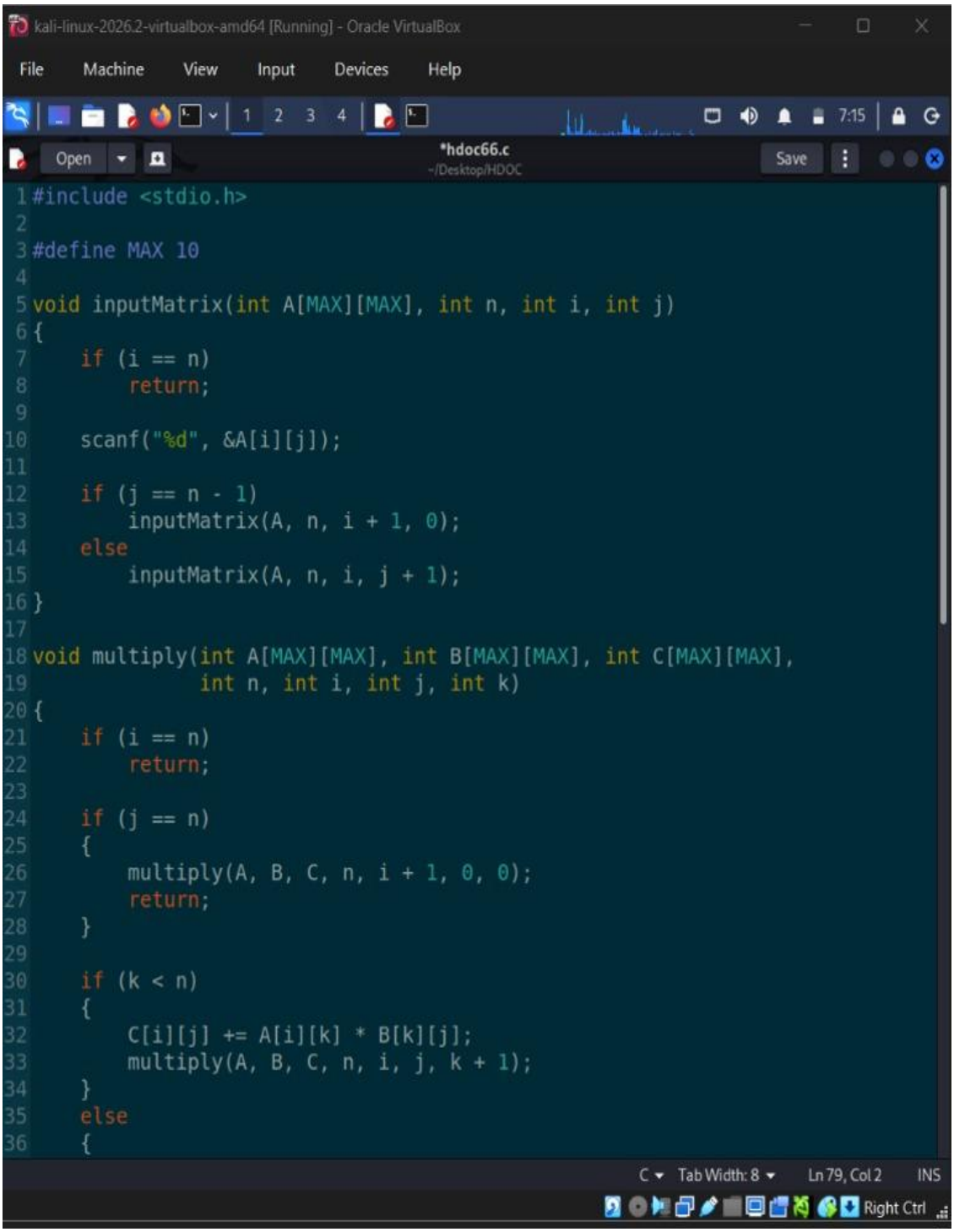
9. Display the resultant matrix recursively.

10. Stop.

Test Case Table

S. No.	Matrix Input	Expected Output	Actual Output	Result
1	n = 1 A = [5] B = [4]	20	20	Pass
2	n = 2 A = 1 2 3 4 B = 5 6 7 8	19 22 43 50	19 22 43 50	Pass
3	n = 3 A = 1 2 3 4 5 6 7 8 9 B = 9 8 7 6 5 4 3 2 1	30 24 18 84 69 54 138 114 90	30 24 18 84 69 54 138 114 90	Pass

C Program




```c
#include <stdio.h>

#define MAX 10

void inputMatrix(int A[MAX][MAX], int n, int i, int j)
{
    if (i == n)
        return;

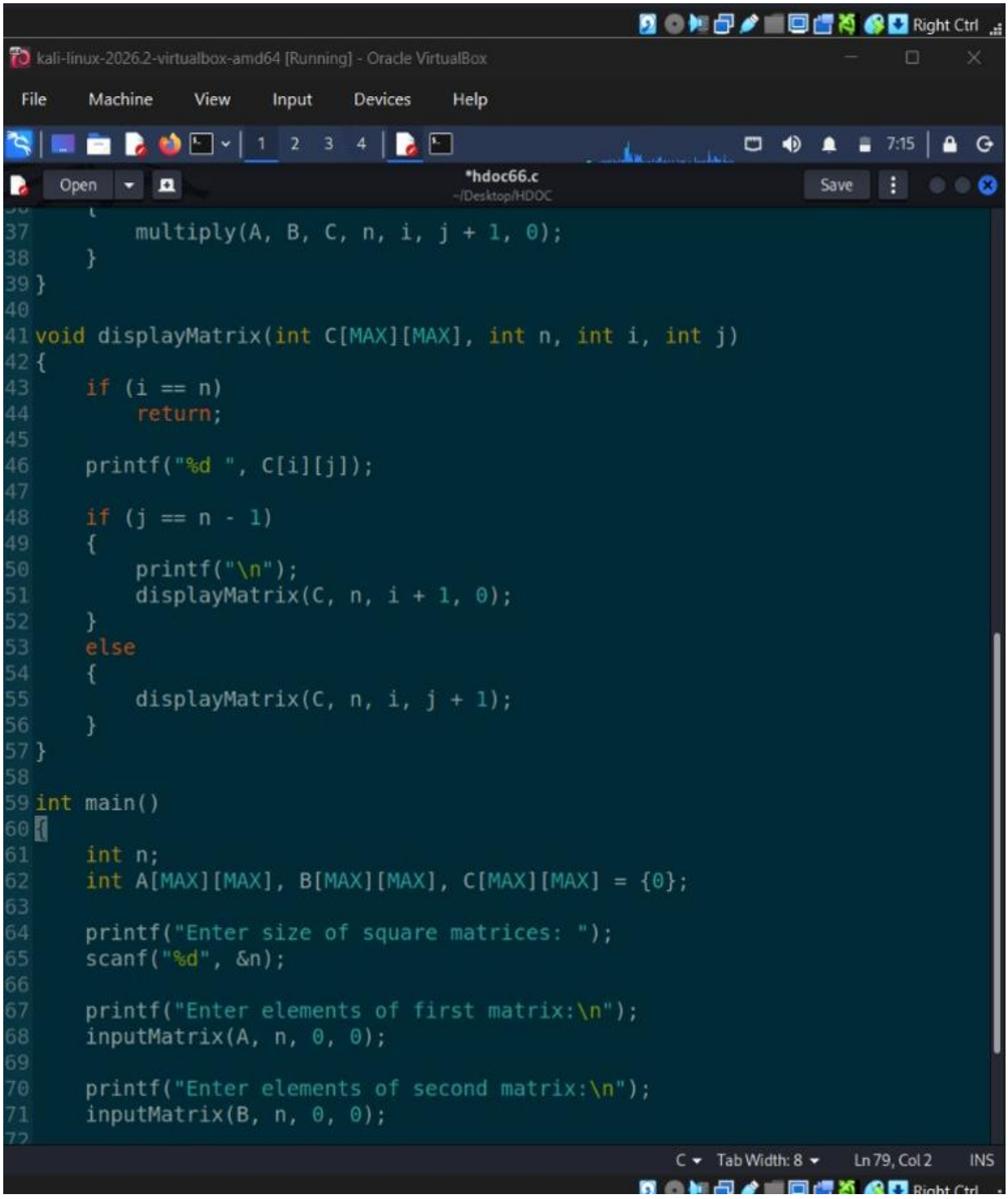
    scanf("%d", &A[i][j]);

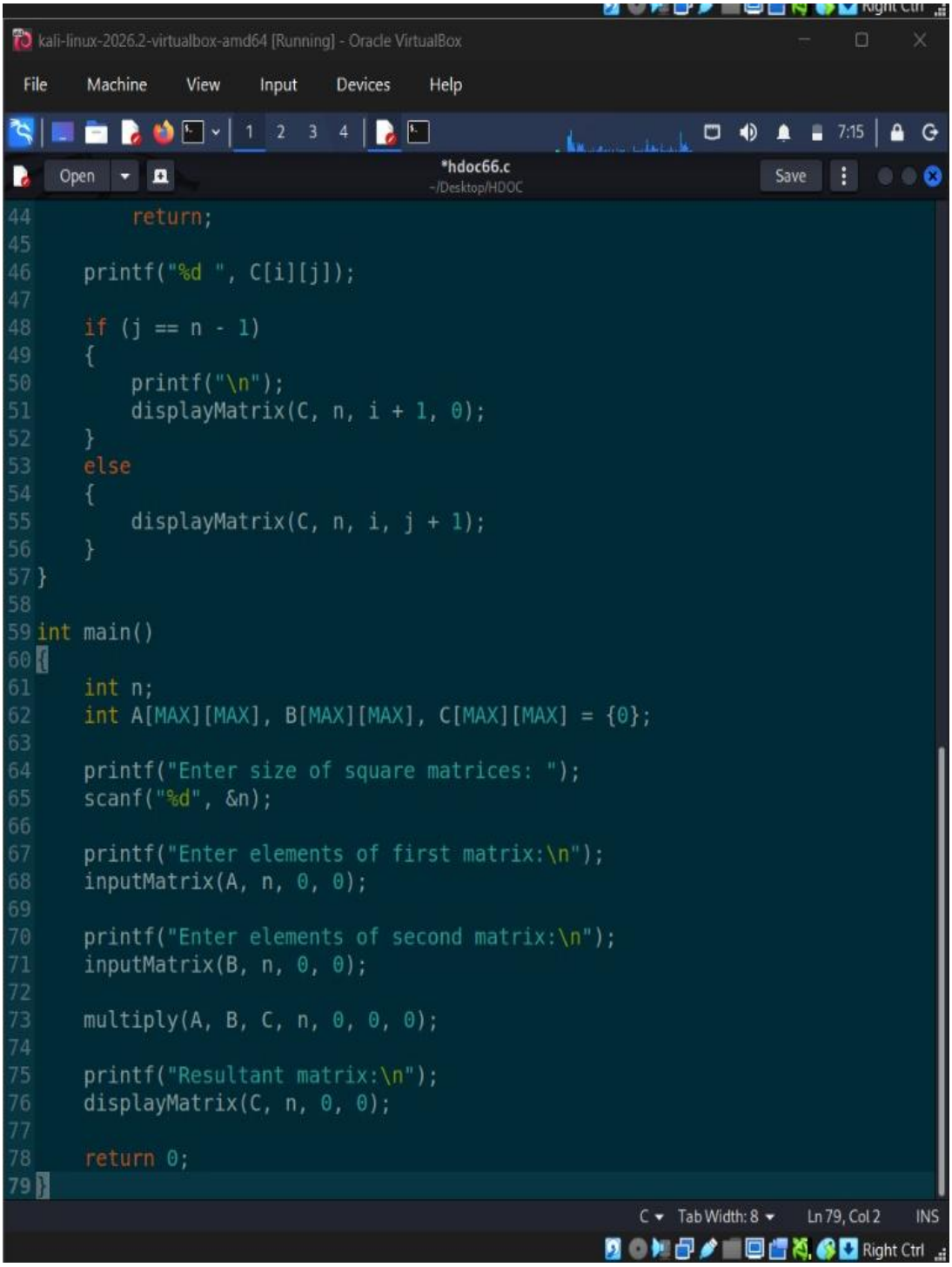
    if (j == n - 1)
        inputMatrix(A, n, i + 1, 0);
    else
        inputMatrix(A, n, i, j + 1);
}

void multiply(int A[MAX][MAX], int B[MAX][MAX], int C[MAX][MAX],
              int n, int i, int j, int k)
{
    if (i == n)
        return;

    if (j == n)
    {
        multiply(A, B, C, n, i + 1, 0, 0);
        return;
    }

    if (k < n)
    {
        C[i][j] += A[i][k] * B[k][j];
        multiply(A, B, C, n, i, j, k + 1);
    }
    else
    {
```


Right Ctrl
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
1 2 3 4
7:15
Open
*hdoc66.c
~/Desktop/HDOC
Save
multiply(A, B, C, n, i, j + 1, 0);
}
}
void displayMatrix(int C[MAX][MAX], int n, int i, int j)
{
if (i == n)
return;
printf("%d ", C[i][j]);
if (j == n - 1)
{
printf("\n");
displayMatrix(C, n, i + 1, 0);
}
else
{
displayMatrix(C, n, i, j + 1);
}
}
int main()
{
int n;
int A[MAX][MAX], B[MAX][MAX], C[MAX][MAX] = {0};
printf("Enter size of square matrices: ");
scanf("%d", &n);
printf("Enter elements of first matrix:\n");
inputMatrix(A, n, 0, 0);
printf("Enter elements of second matrix:\n");
inputMatrix(B, n, 0, 0);
C Tab Width: 8 Ln 79, Col 2 INS
Right Ctrl




```c
        return;

    printf("%d ", C[i][j]);

    if (j == n - 1)
    {
        printf("\n");
        displayMatrix(C, n, i + 1, 0);
    }
    else
    {
        displayMatrix(C, n, i, j + 1);
    }
}

int main()
{
    int n;
    int A[MAX][MAX], B[MAX][MAX], C[MAX][MAX] = {0};

    printf("Enter size of square matrices: ");
    scanf("%d", &n);

    printf("Enter elements of first matrix:\n");
    inputMatrix(A, n, 0, 0);

    printf("Enter elements of second matrix:\n");
    inputMatrix(B, n, 0, 0);

    multiply(A, B, C, n, 0, 0, 0);

    printf("Resultant matrix:\n");
    displayMatrix(C, n, 0, 0);

    return 0;
}
```

Compilation and Execution

Compilation command: gcc hdoc66.c -o hdoc66

```
┌──(kali㉿kali)-[~/Desktop/HDOC]
└─$ gcc hdoc66.c -o hdoc66
```

Execution command: ./hdoc66

Execution Screenshot:

